
Reasoning Pipeline: `proposal_planning` → `CodeAgent` → Final Answer

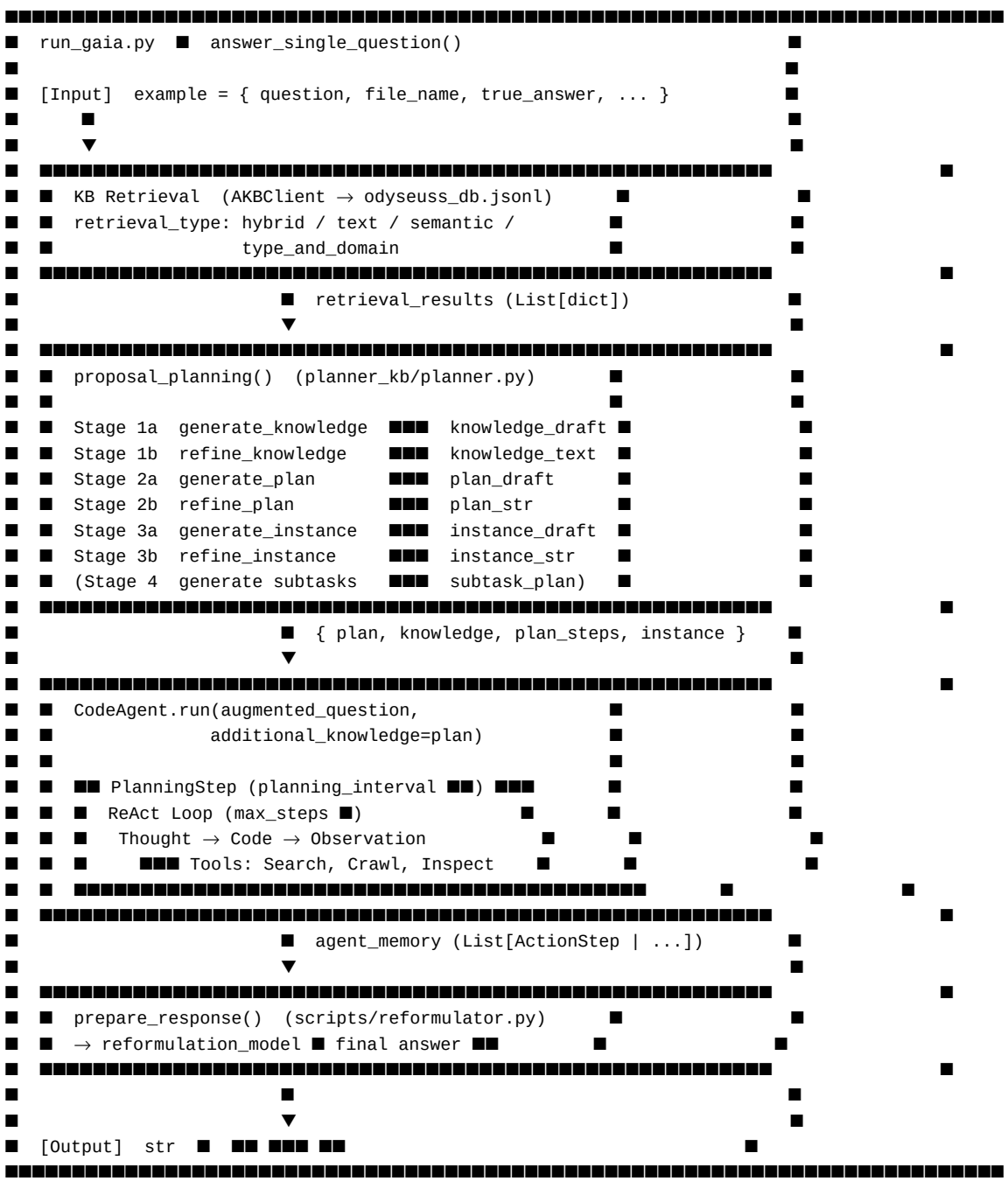
> **■■■**: 2026-04-25

> **■■ ■■■■■**: `examples/open_deep_research/`

> **■■**: Python 3 / smolagents / sentence-transformers

1. ■■ ■■ ■■ (Overview)

■■ ASCII ■■■■ GAIA ■■■■ ■■■ ■■■ ■■■■ ■■ ■■■■■■ ■■■■■.



■■ ■■ ■ ■■ ■■:

■■	■■
`run_gaia.py`	■■ ■■■■■■■■. `answer_single_question()` ■■■
`agent_kb/agent_kb_retrieval.py`	`AgenticKnowledgeBase` + `AKB_Manager` — KB ■■■ ■ ■■
`planner_kb/planner.py`	`proposal_planning()` — 7■■ ■■·■■·■■■■ ■■
`planner_kb/planner_prompts.yaml`	■■ ■■■■ ■■■
`smolagents.CodeAgent`	ReAct ■■ ■■
`scripts/reformulator.py`	■■■■ ■■■ → ■■ ■■ ■■

2. KB ■■ ■■ (KB Retrieval)

2.1 ■■■ ■■ — `odyseuss_db.jsonl`

■ ■■■■ ■■■ ■■ ■■■ ■■■ ■■■■ ■■ ■■■ ■■■ ■■■■.

■■	■■	■■
`task_id`	`str`	■■■ ■■ ID
`task`	`str`	■■■ ■■ ■■■ (■■ `question` ■)
`true_answer`	`str`	■■ ■■■
`task_analysis`	`dict`	`{ knowledge, plan, task_type, domain }`
`agent_planning`	`str | null`	■■■■■ ■■■ ■■ ■■ ■■■
`decision_augmentation`	`dict`	`{ final_reference: { knowledge, plan, instance }, signals_summary: [...] }`
`plan_subtask_action`	`list | null`	`[ { step, subtasks: [{ subtask, subtask_action }] } ]`

`parse_json_file()` ■■■ ■■ ■■ ■ ■■ ■■■■ ■■■■■.

```
# task_analysis.task_type / domain : dict → list ■■
if isinstance(ta_type, dict):
    task_analysis["task_type"] = [ta_type.get("raw"), ta_type.get("normalized")]

# decision_augmentation.final_reference ■ knowledge/plan ■ task_analysis ■ ■■
if not task_analysis.get("knowledge") and fr.get("knowledge"):
    task_analysis["knowledge"] = fr["knowledge"]
```

2.2 ■■ ■■ (Retrieval Modes)

`AKB_Manager` ■■■(`agent_kb_retrieval.py`)■ ■ ■■ ■■ API■ ■■■■■.

`hybrid_search(query, top_k, weights)`

TF-IDF ■■■ ■■■ sentence-transformer ■■ ■■■■ ■■ ■■■■■.

```
score = weights["text"]     × TF-IDF cosine similarity
      + weights["semantic"]  × all-MiniLM-L6-v2 embedding cosine similarity
```

■■ ■■■: `{ "text": 0.5, "semantic": 0.5 }`

`search_by_text(query, field, top_k)`

TF-IDF ■■■■■■■ `task` ■■■■ ■■ ■■■■ ■■■■ ■■. `sklearn.TfidfVectorizer(stop_words="english")` ■■.

`search_by_semantic(query, field, top_k)`

`sentence-transformers/all-MiniLM-L6-v2` ■■■■ ■■■■■ ■ ■■■■ ■■■■■ ■■ ■■.

`type_domain_text_search(query, task_types, domains, top_k, weights)`

type-domain ■■■■ ■■■■ TF-IDF ■■■■ ■■■■ ■■ ■■ ■■:

```
final_score = 0.3 × type_score + 0.3 × domain_score + 0.4 × text_score

type_score   = |query_types ∩ kb_types|   / max(|query_types|, 1)
domain_score = |query_domains ∩ kb_domains| / max(|query_domains|, 1)
```

2.3 ■■ ■■ ■■■■■ ■■■■

`AKB_Manager.hybrid_search()` / `type_domain_text_search()` ■ ■■ ■■:

```
{
    "task_id":           str,
    "total_score":       float,    # hybrid: TF-IDF + semantic ■■
    # (type_domain: "score" + "overlap_count")
    "task":              str,
    "true_answer":       str,
    "agent_planning":    str | None,
    "agent_experience":  str | None,
    "task_analysis": {
        "knowledge": str,
        "plan":      list[str],
        "task_type": list[str],
        "domain":    list[str],
    },
    "plan_subtask_action": list | None,
    "instance":          str | None,   # decision_augmentation.final_reference.instance
    "decision_guide":    list | None,  # decision_augmentation.signals_summary
}
```

`decision_guide` ■■ ■■■ ■■:

```
{
    "level":       "knowledge" | "plan" | "instance",
    "failures":    list[str],   # ■■ ■■ "failure": str
    "causes":      list[str],   # ■■ ■■ "cause": str
    "corrections": list[str],   # ■■ ■■ "correction": str
}
```

3. `proposal_planning` ■■ (Proposal Planning)

3.0 ■■ ■■■■

```
def proposal_planning(
    example,                          # GAIA ■■■ dict (question, file_name, ...)
    augmented_question: str,          # ■■ ■■■ ■■ ■■■ ■■ ■■■
    model_name: str,
    key: str,
    url: str,
    model: Any,
    slm: bool,
    retrieval_method: Callable,       # AKBClient.hybrid_search ■
    top_k: int,
```

```
    retrieval_option: Literal["task_text", "type_and_domain"] = "task_text",
    type_domain_retrieval_method=None,
    plan_mode: Optional[str] = None,  # "plan" | "subtask" | "plan_subtask" | ...
    tools=None,
    managed_agents=None,
    planning_prompt_templates=None,
) -> dict
```

■■■:

```
{
    "plan":             str,   # CodeAgent ■ ■■■ ■■ ■■■■
    "knowledge":        str,   # refined knowledge text
    "plan_steps":       str,   # refined plan (bullet list)
    "instance":         str,   # refined instance
    "retrieval_results": list,
    "examples":         dict,  # subtask_examples dict
}
```

3.1 ■■ [1] — ■■ ■■■ ■■ (Retrieve Similar Tasks)

■■: ■■ ■■■ ■■ ■■■ KB ■■■ `top_k`■■ ■■■ ■■ ■■■ few-shot ■■■ Decision Guide■ ■■.

■■:

- `example["question"]` — ■■ ■■ ■■■
- `retrieval_option` — `"task_text"` ■■ `"type_and_domain"`

■■ ■■:

```
if retrieval_option == "type_and_domain":
    # 1) LLM■■ ■■ ■■■■ ■■
    classification = classify_task_type_and_domain(task=augmented_question, ...)
    task_types = classification["task_type_normalized"]
    domains    = classification["domain_normalized"]
    # 2) type+domain+text ■■ ■■■ ■■
    retrieval_results = type_domain_retrieval_method(
        example["question"], task_types=task_types, domains=domains, top_k=top_k
    )
    # fallback: ■■ ■■■ text ■■■■ ■■
else:
    retrieval_results = retrieval_method(example["question"], top_k=top_k)
```

■■ ■ ■■■ ■■ ■■:

```
knowledge_examples = build_similar_task_direction_blocks(retrieval_results)
plan_examples      = build_similar_task_plan_blocks(retrieval_results)
instance_examples  = build_instance_examples(retrieval_results)
guide_knowledge    = build_decision_guide_blocks(retrieval_results, level="knowledge")
guide_plan         = build_decision_guide_blocks(retrieval_results, level="plan")
guide_instance     = build_decision_guide_blocks(retrieval_results, level="instance")
```

■■: `retrieval_results` (List[dict]) + 6■■ ■■·■■■ ■■■

3.2 ■■ [2] — Stage 1a: `generate_knowledge`

■■: ■■■ ■■■■ ■■ ■■ ■■■ ■■■ ■■(■■■ + ■■■)■ ■■.

■■ ■■■■:

```
def generate_knowledge(
    task: str,
    examples: str,           # knowledge_examples ■■
```

```
      planning_prompt_template: dict,
      model_name: str, key: str, url: str, model: Any, slm: bool,
  ) -> str
```

■■■■ ■■■ (`planner_prompts.yaml` — `knowledge_prompt`):

```
  knowledge_prompt: |
    Extract the domain knowledge required to understand and solve the given task.
    Provide both:
        (a) declarative knowledge: domain concepts and definitions used
        (b) procedural knowledge: methodology, paradigm, or algorithm to apply
    Return only the knowledge as plain text. Do NOT include the direct answer.

    TASK:
    {{task}}

    SIMILAR TASKS:
    {{examples}}
```

LLM ■■: `call_model(query=prompt, model_name, key, url, model, slm)`

■■: `knowledge_draft` (str) — ■■■·■■■ ■■ ■■■

3.3 ■■ [3] — Stage 1b: `refine_knowledge` (with Decision Guide)

■■: Stage 1a■ ■■ ■■■ Decision Guide ■■■ ■■. `decision_guide`■ ■■ ■■■ draft■ ■■■ ■■.

■■ ■■■■:

```
  def refine_knowledge(
      task: str,
      draft: str,              # generate_knowledge ■ ■■
      decision_guide: str,     # guide_knowledge ■■
      planning_prompt_template: dict,
      model_name: str, key: str, url: str, model: Any, slm: bool,
  ) -> str
```

■■■■ ■■■ (`planner_prompts.yaml` — `refine_knowledge_prompt`):

```
  refine_knowledge_prompt: |
    You generated domain knowledge for a task. A Decision Guide identifies failure patterns
    observed in similar tasks.
    Refine the generated knowledge to address the failures. Only modify what the guide says
    is insufficient or wrong.
    Preserve all correct parts. Return the full refined knowledge as plain text.

    TASK:
    {{task}}

    GENERATED KNOWLEDGE:
    {{draft}}

    DECISION GUIDE:
    {{decision_guide}}
```

■■: `knowledge_text` (str) — ■■■ ■■ ■■

3.4 ■■ [4] — Stage 2a: `generate_plan`

■■: ■■■ ■■ ■■■ ■■■ ■■■■ ■■ 10■■ ■■■ ■■■ ■■.

■■ ■■■■:

```
  def generate_plan(
      task: str,
      knowledge: str,          # knowledge_text
      examples: str,           # plan_examples ■■
      planning_prompt_template: dict,
      model_name: str, key: str, url: str, model: Any, slm: bool,
  ) -> str
```

■■■■ ■■■ (`planner_prompts.yaml` — `plan_prompt`):

```
  plan_prompt: |
    Develop a high-level plan of no more than 10 steps to solve the given task based only
    on the provided knowledge and similar tasks.
    The plan must not include the final answer and the final conclusion.

    Requirements:
    - Use the knowledge explicitly and progressively in each step.
    - Derive a decision criterion: an intermediate, task-specific quantity, rule, or structure
      that can be used to select the answer.

    Strict Output Format:
    - <step>
    - <step>
    - ...

    TASK:
    {{task}}

    KNOWLEDGE:
    {{knowledge}}

    SIMILAR TASKS:
    {{examples}}
```

■■: `plan_draft` (str) — bullet-list ■■■ ■■■ ■■

3.5 ■■ [5] — Stage 2b: `refine_plan` (with Decision Guide)

■■: Stage 2a■ ■■ ■■■ Decision Guide ■■■ ■■.

■■ ■■■■:

```
  def refine_plan(
      task: str,
      knowledge: str,
      draft: str,              # plan_draft
      decision_guide: str,     # guide_plan ■■
      planning_prompt_template: dict,
      model_name: str, key: str, url: str, model: Any, slm: bool,
  ) -> str
```

■■■■ ■■■ (`planner_prompts.yaml` — `refine_plan_prompt`):

```
  refine_plan_prompt: |
    You generated a solution plan for a task. A Decision Guide identifies failure patterns
    observed in similar tasks.
    Refine the generated plan to address the failures. Only modify steps that the guide says
    are wrong or missing.
    Preserve all correct steps. Return the full refined plan as a bullet list.

    TASK:
    {{task}}

    KNOWLEDGE:
```

```
    {{knowledge}}

    GENERATED PLAN:
    {{draft}}

    DECISION GUIDE:
    {{decision_guide}}
```

■■: `plan_str` (str) — ■■■ ■■ ■■

3.6 ■■ [6] — Stage 3a: `generate_instance`

■■: ■■■ ■■ ■■■■■ ■■■■■ ■■■ ■■ ■■■■■ ■■. ■■ ■■■■ ■■ ■■■ ■■■■ ■■ ■■ ■■.

■■ ■■■■:

```
  def generate_instance(
      task: str,
      knowledge: str,
      plan: str,               # plan_str
      examples: str,           # instance_examples ■■
      planning_prompt_template: dict,
      model_name: str, key: str, url: str, model: Any, slm: bool,
  ) -> str
```

■■■■ ■■■ (`planner_prompts.yaml` — `instance_prompt`):

```
  instance_prompt: |
    Generate a concrete execution instance that grounds the given plan into specific,
    actionable steps with actual values and reasoning.
    The instance should demonstrate exactly how to apply the plan to THIS task, showing
    intermediate computations and logical deductions.

    Requirements:
    - Walk through each plan step with concrete values from the task.
    - Show actual computations, lookups, or reasoning steps.
    - Do NOT reveal or guess the final answer — stop just before the conclusion.

    TASK:
    {{task}}

    KNOWLEDGE:
    {{knowledge}}

    PLAN:
    {{plan}}

    SIMILAR INSTANCES:
    {{examples}}
```

■■: `instance_draft` (str) — ■■■ ■■ ■■■■ ■■■

3.7 ■■ [7] — Stage 3b: `refine_instance` (with Decision Guide)

■■: Stage 3a■ ■■■■ ■■■ Decision Guide■ ■■.

■■ ■■■■:

```
  def refine_instance(
      task: str,
      knowledge: str,
      plan: str,
      draft: str,              # instance_draft
```

```
    decision_guide: str,     # guide_instance ■■
    planning_prompt_template: dict,
    model_name: str, key: str, url: str, model: Any, slm: bool,
) -> str
```

■■■■ ■■■ (`planner_prompts.yaml` — `refine_instance_prompt`):

```
refine_instance_prompt: |
  You generated a concrete execution instance for a task. A Decision Guide identifies
  failure patterns observed in similar tasks.
  Refine the instance to fix the failures identified in the guide. Only modify what is
  incorrect or incomplete.
  Preserve all correct parts. Return the full refined execution instance.

  TASK:
  {{task}}

  KNOWLEDGE:
  {{knowledge}}

  PLAN:
  {{plan}}

  GENERATED INSTANCE:
  {{draft}}

  DECISION GUIDE:
  {{decision_guide}}
```

■■: `instance_str` (str) — ■■■ ■■ ■■■■

3.8 ■■ [■■] — Stage 4: Subtask ■■

`plan_mode`■ `"subtask"`, `"plan_subtask"`, `"plan_subtask_action"` ■ ■■■■ `planning_prompt_templates`■ ■■■ ■■ ■■■■■■.

```
if plan_mode in ("subtask", "plan_subtask", "plan_subtask_action"):
    stage2_prompt = populate_template(
        planning_prompt_templates["initial_plan_subtask_action_stage2"],
        variables={
            "task":        augmented_question,
            "tools":       tools,
            "managed_agents": managed_agents,
            "plan_steps":  plan_str,
            "examples":    subtask_examples[example_key],
        },
    )
    subtask_plan = call_model(query=stage2_prompt, ...)
```

■■ `plan` ■■■ ■■:

```
# plan_mode ■■ (■■)
final_plan = f"Knowledge:\n{knowledge_text}\n\nPlan:\n{plan_str}\n\nInstance:\n{instance_str}"

# plan_mode ■■ (subtask ■■)
final_plan = f"Knowledge:\n{knowledge_text}\n\nPlan:\n{plan_str}\n\nInstance:\n{instance_str}\n\nSubtasks:\n{subtask
```

4. CodeAgent ■■ (CodeAgent Execution)

4.1 ■■■■ ■■

`create_agent_hierarchy()` ■■■■ `CodeAgent`■ ■■■■■■.

```
manager_agent = CodeAgent(
    model=model,
    tools=[],
    max_steps=args.max_steps,          # ■■ 12
    verbosity_level=2,
    additional_authorized_imports=AUTHORIZED_IMPORTS,
    planning_interval=args.planning_interval,  # ■■ 1
    managed_agents=[],
    debug=debug,
    agent_kb=args.agent_kb,
    top_k=args.top_k,
    retrieval_type=args.retrieval_type,
    plan_mode=args.plan_mode,
)
```

`AUTHORIZED_IMPORTS`■■ `requests`, `pandas`, `numpy`, `sympy`, `sklearn`, `scipy`, `PIL`, `PyPDF2`, `torch` ■ 25■ ■■■■■■ ■■■■■.

4.2 `additional_knowledge` ■■

`proposal_planning()`■ ■■■ `plan` ■■■■■ `"plan"` ■ ■(■■ `plan_mode_planning()` ■■ ■■■)■ `additional_knowledge`■ ■■■■■ ■■■■■.

```
# run_gaia.py
if args.kb_type == "proposal":
    def planner_fn(task, tools, managed_agents) -> dict:
        return proposal_planning(
            example=example,
            augmented_question=augmented_question,
            ...
            plan_mode=args.plan_mode,
            tools=tools,
            managed_agents=managed_agents,
            planning_prompt_templates=agent.prompt_templates["planning"],
        )
    agent.planner_fn = planner_fn

# ■■■■ ■■
final_result = agent.run(
    augmented_question,
    additional_knowledge=_additional_knowledge_for_planning
)
```

`additional_knowledge`■ `CodeAgent`■ ■■■ ■■■■ ■■■■ ■■■■ ■■■■■ ■■ ■■ ■ ■■■ ■■ ■■■ ■■■■■. ■■■■■ ■■■■ ■■■ ■■■ ■■■■.

```
Knowledge:
<refined knowledge text>

Plan:
<refined plan bullet list>

Instance:
<refined execution instance>
```

4.3 ReAct ■■ (Thought → Code → Observation)

`CodeAgent`■ `smolagents` ■■■■■■ ReAct ■■■■■■ ■■■■■.

```
Iteration 1 ~ max_steps:
```

```
    ■■ PlanningStep (planning_interval ■■ ■■) ■■■■■■■■■■■■■■■
    ■  LLM■ ■■ ■■■■ ■■ high-level plan ■■■■           ■
    ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    ■■ ActionStep ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    ■  Thought : LLM■ ■■ ■■■■ Python ■■■■ ■■           ■
    ■  Code    : ■■■■■■ Python ■■ ■■                    ■
    ■  Observation: ■■ ■■ ■■(stdout / ■■■■) ■■         ■
    ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
         ■
         ▼ (max_steps ■■ ■■ final_answer() ■■ ■ ■■)
```

`planning_interval=1` ■■ ■ ■■■■■ PlanningStep■ ■■■■■■.

4.4 ■■ ■■■■ ■■ (Tools)

```
| ■■ | ■■■■ | ■■ |
```

|---|---|---|

```
| `SearchTool` | `scripts/searcher.py` | Tavily / Exa ■ ■ ■■ |
| `CrawlerReadTool` | `scripts/async_web_crawler.py` | URL ■■ ■■■■ |
| `CrawlerArchiveSearchTool` | `scripts/async_web_crawler.py` | Wayback Machine ■■■■■ ■■ |
| `TextInspectorTool` | `scripts/text_inspector_tool.py` | PDF/■■■■ ■■ ■■ ■■ |
| `AudioInspectorTool` | `scripts/audio_inspector_tool.py` | ■■■■ ■■ ■■ ■■ |
| `VisualInspectorTool` | `scripts/visual_inspector_tool.py` | ■■■■ ■■ ■■ ■■ |
```

■■■■■ `create_agent_hierarchy()` ■■ ■■ ■■■■■■■■, `CodeAgent.tools` ■■■■■ ■■■■■■.

4.5 ■■■■ ■■

■■■■■ ■■■■■ ■ ■■ `smolagents.memory` ■■■■■ ■■■■■■.

```
| ■■ | ■■ |
```

|---|---|

```
| `TaskStep` | ■■■■■■ ■■■■ ■■■■ ■■■■■ `additional_knowledge` |
| `PlanningStep` | `planning_interval`■■ LLM■ ■■■■■ ■■■■ ■■ |
| `ActionStep` | ■ ■■■■ (Thought, Code, Observation) ■■ ■ |
```

■■ ■■ ■ `agent.write_memory_to_messages(summary_mode=True)`■ ■■ ■■■■■ ■■■■ ■■■■■ ■■■■■■■.

5. ■■ ■■ ■■ (Final Answer Extraction)

5.1 `prepare_response()` — `scripts/reformulator.py`

```
def prepare_response(
    original_task: str,
    inner_messages,          # agent.write_memory_to_messages() ■■
    reformulation_model: Model,
    multiple: bool = False,
) -> str
```

■■ ■■:

1. ■■■■ ■■■■■ ■■ ■■■■■ ■■
2. ■■■■■ ■■■■(`inner_messages`)■ USER ■■■■ ■■■■■ ■■ ■■ ■■

3. ■■ ■■ ■■■■ ■■ — ■■■■ ■■ ■■ ■■

■■ ■■ ■■■■ ■■:

```
Read the above conversation and output a FINAL ANSWER to the question. The question
is repeated here for convenience:

{original_task}

FINAL ANSWER FORMAT: Your response must strictly follow these formatting rules:
- For NUMBERS: Use digits only (not words), omit commas and units (no $, USD, %, etc.)
  unless specifically requested
- For TEXT: Omit articles and abbreviations unless specified, exclude final punctuation (.!?)
- For LISTS: Provide comma-separated values following the above number/text rules
- Follow ALL formatting instructions in the original question (alphabetization, sequencing,
  decimal places, etc.)
- Please carefully understand the requirements of the original task and ensure that the
  final output meets the specific units given in the question (/Angstrom, /thousand hours, etc.)
- If you cannot determine an answer, respond only with: "Unable to determine"
- Your entire response should consist of ONLY the requested information in the EXACT format
  specified - nothing more, nothing less.
```

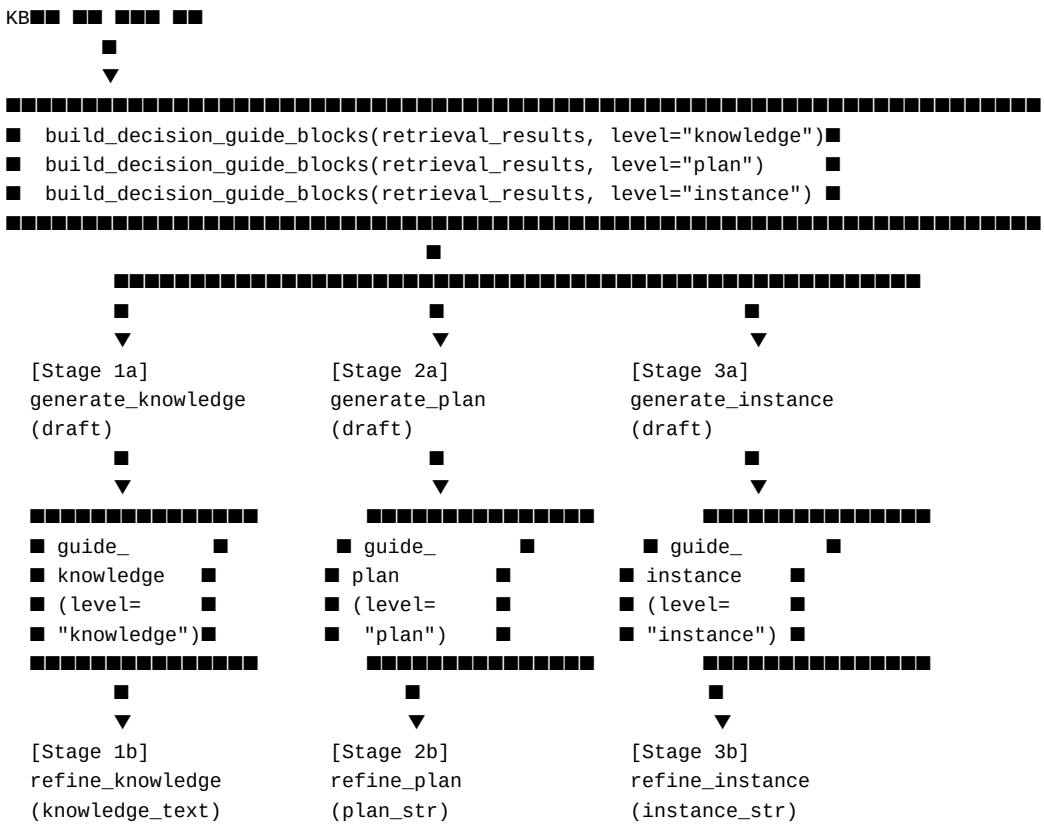
4. `reformulation_model(messages).content` ■■

5. `response.split("FINAL ANSWER: ")[-1].strip()` ■ ■■ ■ ■■

■■: `str` — ■■·■■■·■■■ ■■■ ■■■ ■■■ ■■ ■■

6. Decision Guide ■■ ■■ (Decision Guide Refine Flow)

6.1 ■■ ■■■



6.2 Decision Guide ■■ ■■

```
# decision_guide ■■ ■■■ ■■
{
    "level":       "knowledge" | "plan" | "instance",
    "failures":    ["■■ ■■ ■■ 1", "■■ ■■ ■■ 2", ...],
    "causes":      ["■■ ■■ 1", "■■ ■■ 2", ...],
    "corrections": ["■■ ■■ 1", "■■ ■■ 2", ...],
}
```

`build_decision_guide_blocks()` ■■■ ■■ ■■■■■ ■■■ ■■■■■ ■■■■■■:

```
[From Task: <task text>]
  [KNOWLEDGE] Failure: <failures[0]>
              Cause: <causes[0]>
              Correction: <corrections[0]>
  [PLAN] Failure: <failures[1]>
         ...
```

6.3 Generate-then-Refine ■■■ ■■ (SLM ■■)

```
| ■■ | ■■ |
```

|---|---|

```
| **■■ ■■ ■■** | SLM■ ■ Guide ■■■ ■■■■■ ■■ ■■ ■■■■■ ■■■ ■■, ■■ ■■ ■■ |
```

```
| **Generate-then-Refine** | ■■ ■■ ■■ ■■■ ■■■ ■■ → ■■ ■■ ■■■ ■■ ■■■■■■ ■■ |
```

■ ■■■ SLM ■■■ ■■■ ■■■ ■■■ ■■ ■■■■■ ■■, ■ ■■ ■■ LLM ■■■ ■■■ ■■■■■■ ■■■■■. ■ refine ■■■ `if not decision_guide: return draft`■ ■■■■■, KB■ ■■ ■■■ ■■■■■ ■■ ■■ ■■■■■ LLM ■■■ ■■■■■■.

7. ■■■ ■■■ (Data Schemas)

7.1 `odyseuss_db.jsonl` ■■■ ■■■

```
{
  "task_id":        "string",
  "task":           "string",
  "true_answer":    "string",
  "agent_planning": "string | null",
  "task_analysis": {
    "knowledge":  "string",
    "plan":       ["step1", "step2", "..."],
    "task_type":  ["computation", "analysis", "..."],
    "domain":     ["mathematics", "science", "..."]
  },
  "plan_subtask_action": [
    {
      "step": "string",
      "subtasks": [
        {
          "subtask":        "string",
          "subtask_action": "string"
        }
      ]
    }
  ],
  "decision_augmentation": {
    "final_reference": {
      "knowledge": "string",
      "plan":      ["step1", "..."],
```

```
      "instance":  "string"
    },
    "signals_summary": [
      {
        "level":       "knowledge | plan | instance",
        "failures":    ["string"],
        "causes":      ["string"],
        "corrections": ["string"]
      }
    ]
  }
}
```

7.2 ■■ ■■ ■■■■ ■■■ (Retrieval Result Dict)

`AKB_Manager.hybrid_search()` ■■ ■■■■ ■ ■■:

```
{
    "task_id":           str,
    "total_score":       float,     # hybrid / type_domain ■■
    "task":              str,
    "true_answer":       str,
    "agent_planning":    str | None,
    "agent_experience":  str | None,
    "task_analysis": {
        "knowledge": str | None,
        "plan":      list[str] | None,
        "task_type": list[str],
        "domain":    list[str],
    } | None,
    "plan_subtask_action": list | None,
    "instance":          str | None,
    "decision_guide": [
        {
            "level":       str,    # "knowledge" | "plan" | "instance"
            "failures":    list[str],
            "causes":      list[str],
            "corrections": list[str],
        }
    ] | None,
}
```

7.3 `proposal_planning()` ■■ ■■■■ ■■■

```
{
    "plan": str,
    # ■■ (plan_mode == None ■■ "plan"):
    #   "Knowledge:\n{knowledge_text}\n\nPlan:\n{plan_str}\n\nInstance:\n{instance_str}"
    # subtask ■■:
    #   "Knowledge:...\n\nPlan:...\n\nInstance:...\n\nSubtasks:\n{subtask_plan}"

    "knowledge":         str,     # Stage 1b ■■ (refined knowledge)
    "plan_steps":        str,     # Stage 2b ■■ (refined plan bullet list)
    "instance":          str,     # Stage 3b ■■ (refined instance)
    "retrieval_results": list,    # KB ■■ ■■ ■■
    "examples": {
        "plan_subtask":        str,   # build_plan_subtask_examples() ■■
        "plan_subtask_action": str,   # build_plan_subtask_action_examples() ■■
    },
}
```

■■ A. ■■■ ■■ ■■■■ (Task Classification Prompt)

`retrieval_option == "type_and_domain"` ■ ■ ■■■■ ■■ ■■■■■■■■.

```
  task_classification_prompt: |
    Given the task below, select all applicable task types and domains from the predefined lists.
    Choose only values that appear in the provided lists. You may select multiple values.

    TASK_TYPES (choose one or more):
    {{task_types}}

    DOMAINS (choose one or more):
    {{domains}}

    Return STRICT JSON (no extra keys, no prose outside the JSON):
    {
        "task_type_normalized": ["<type1>", ...],
        "domain_normalized": ["<domain1>", ...]
    }

    TASK:
    {{task}}}
```

■■ ■■■ ■■ ■■:

- **TASK_TYPE_CONSTANTS**: `algebra`, `algorithm`, `analysis`, `approximation`, `calculation`, `computation`, `decision_support`, `engineering`, `geometry`, `simulations`
- **DOMAIN_CONSTANTS**: `arts_humanities`, `business`, `computer_science`, `engineering`, `law`, `mathematics`, `medicine`, `science`, `social_sciences`, `technology`

■■ B. ■■ ■■ ■■ ■■

■■	■■ ■■
■■ ■■ ■■■	`examples/open_deep_research/run_gaia.py`
KB ■■■·■■	`examples/open_deep_research/agent_kb/agent_kb_retrieval.py`
Proposal Planning ■■	`examples/open_deep_research/planner_kb/planner.py`
■■■■ ■■■	`examples/open_deep_research/planner_kb/planner_prompts.yaml`
Planner ■■ API	`examples/open_deep_research/planner_kb/__init__.py`
■■ ■■ ■■	`examples/open_deep_research/scripts/reformulator.py`